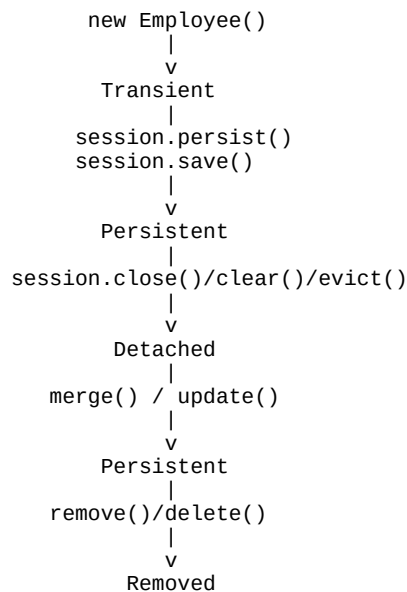


Chapter 4 - Hibernate Entity Lifecycle

1. Introduction

Every Hibernate entity passes through different lifecycle states. Understanding these states is essential for interviews and debugging persistence issues.

2. Entity Lifecycle Diagram



3. Transient State

A newly created Java object that is not associated with a Session. It has no database row and is not managed by Hibernate.

4. Persistent State

After calling `persist()` or `save()`, the entity becomes Persistent. Hibernate manages it inside the current Session (Persistence Context). Changes are tracked automatically using dirty checking.

5. Detached State

When the Session is closed, cleared, or the entity is evicted, the entity becomes Detached. Hibernate no longer tracks changes made to it.

6. Removed State

Calling `remove()` or `delete()` marks the entity for deletion. The DELETE SQL is executed during flush/commit.

7. State Transition Example

```
Employee emp = new Employee();    // Transient
session.persist(emp);              // Persistent
session.close();                  // Detached

Session s2 = sessionFactory.openSession();
s2.merge(emp);                    // Persistent again
s2.remove(emp);                  // Removed
```

8. SQL Generated

```
INSERT INTO EMPLOYEE (...);

UPDATE EMPLOYEE
SET NAME = ?
WHERE ID = ?;

DELETE FROM EMPLOYEE
WHERE ID = ?;
```

9. update() vs merge()

update() reattaches the same detached object and can throw an exception if another instance with the same identifier is already managed.

merge() copies the detached object's state into a managed entity and returns the managed instance.

10. Interview Questions

1. What are the four entity states? 2. What is a detached object? 3. What is the Persistence Context? 4. Difference between update() and merge()? 5. What is dirty checking?